

Quiz 1 (Version 4)

CS 320: Concepts of Programming Languages

September 16, 2026

Name: Nathan Mull

BUID: 12345678

- You will have approximately 30 minutes to complete this quiz. Make sure to read every question; some are easier than others.
- Make very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).
- We will not look at any work on the pages marked *"This page is intentionally left blank."*. You should use these pages for scratch work.

1 OCaml Programming

(10 points) Implement the function

```
val rev_muls : int -> int -> int list
```

so that `rev_muls k n` is the list of all positive multiples of `k` at most `n` in decreasing order. The behavior of the function is undefined if `k` is not positive.

```
let _ = assert (rev_muls 3 16 = [15;12;9;6;3])
let _ = assert (rev_muls 2 6 = [6;4;2])
let _ = assert (rev_muls 16 3 = [])
```

You may not use any functions from the standard library except for arithmetic operators and list constructors (i.e., `[]` and `(::)`).

```
let rev_muls k n =
  let rec go i =
    if i > 0
    then i :: go (i - k)
    else []
  in go (n - n mod k)
     or  $n / k * k$ 
```

2 Types and Values

(10 points) Determine the type of the function `foo` and the value of `bar` below.

```
let rec foo l =  
  match l with  
  | [] -> []  
  | [x] -> []  
  | x :: y :: xs -> [x;y] :: foo (y :: xs)  
let bar = foo [1;2;3;4;5;6]
```

Justify your answers.

`val foo : int list → int list list`

technically the type is
`'a list → 'a list list`

$$\begin{aligned} \text{foo } [1;2;3;4;5;6] &= \\ [1;2] :: \text{foo } [2;3;4;5;6] &= \\ [1;2] :: [2;3] :: \text{foo } [3;4;5;6] &= \\ [1;2] :: [2;3] :: [3;4] :: \text{foo } [4;5;6] &= \\ [1;2] :: [2;3] :: [3;4] :: [4;5] :: \text{foo } [5;6] &= \\ [1;2] :: [2;3] :: [3;4] :: [4;5] :: [5;6] :: \text{foo } [] &= \\ [1;2] :: [2;3] :: [3;4] :: [4;5] :: [5;6] :: [] \end{aligned}$$

Note. This function just joins adjacent integers into a list

This page is intentionally left blank.